

Day 12

Managing Users and Permissions in Linux

Link Count, Hard and Soft Links, and sudo: explained in simple language

What you will learn	Simple meaning
Inode and link count	How Linux keeps track of how many names a file has
Directory link count	Why a folder shows 2, 3, 4 ... in ls -l
Hard and soft links	Two ways to make a second name or shortcut for a file
sudo	How to run admin commands safely, and how to give a user sudo power

Tip: Practice in your home folder or a practice VM. Create test files only, never link or delete real system files.

1. Simple Terms You Must Know

Term	Meaning in simple words
Inode	A small record that stores everything about a file (size, owner, permissions, where data is on disk) except the file name . Every file has a unique inode number.
Link	A name that points to an inode.
Link count	How many names (links) point to the same inode. It is the number shown in the 2nd column of <code>ls -l</code> .
Hard link	Another name for the same file (same inode).
Soft link (symbolic link)	A shortcut file that stores the path of another file. Like a shortcut on a desktop.
File system	The way a disk partition is organised to store files (for example ext4).
Parent directory	The folder that contains the current folder.
Privilege escalation	Temporarily getting higher power (like root) to do an admin task.
Audit trail	A written record of who did what. Useful for finding mistakes or misuse.

Easy picture: Think of an inode as a **house** and the file names as **address labels** that lead to it. A house can have more than one label, but there is still only one house.

2. Link Count Basics

The **link count** is the number of references (names) pointing to one inode. You see it with `ls -l`, in the column right after the permissions.

Syntax: `ls -l [file or folder]`

```
$ ls -l
drwxr-xr-x  2 user group 4096 Dec 18 10:00 example-dir
-rw-r--r--  1 user group   45 Dec 18 10:00 example-file
```

Item	Link count	Why
example-file	1	A new file has just one name pointing to its inode.
example-dir	2	An empty folder has two names: its own name inside the parent, and the <code>.</code> inside itself.

Remember: Rule: every file starts with link count **1**. Every folder starts with link count **2** (so a folder never shows less than 2).

3. Link Count for Directories

Every folder contains two special entries:

Entry	Meaning
.	The current directory (the folder itself).
..	The parent directory (one level up).

Formula: link count of a folder = **2 + number of subfolders inside it**. Each subfolder has a .. entry that points back to its parent, so each subfolder adds 1 to the parent's count.

```
$ mkdir dir1
$ mkdir dir1/subdir1
$ ls -ld dir1
drwxr-xr-x  3 user group 4096 Dec 18 10:00 dir1
```

New options and commands used:

Command	Meaning
mkdir name	Make directory: create a folder. Syntax: mkdir [options] foldername
ls -ld name	The -d option shows details of the folder itself , not the files inside it.

Why is dir1's count 3? Three names point to the dir1 inode:

- **dir1**: its name inside the parent folder.
- **dir1/.**: the dot entry inside dir1 itself.
- **dir1/subdir1/..**: the double-dot entry inside the subfolder, pointing back up to dir1.

```
# Add one more subfolder and watch the count grow
$ mkdir dir1/subdir2
$ ls -ld dir1
drwxr-xr-x  4 user group 4096 Dec 18 10:05 dir1
```

Small correction: Your original notes say the three links are '.' (itself), '..' (parent) and 'subdir1'. The count is right (3), but the third link is really the '..' inside subdir1, not the name subdir1 itself. The formula '2 + subfolders' is the easy way to remember it.

Remember: Files inside a folder do **not** change the folder's link count. Only **subfolders** do.

4. Link Count for Files and Hard Links

Use of ln: link: creates another name for a file.

Syntax (hard link): ln existing_file new_name

```
$ echo "hello" > file1
$ ln file1 file2
$ ls -l
-rw-r--r--  2 user group 6 Dec 18 10:00 file1
-rw-r--r--  2 user group 6 Dec 18 10:00 file2
```

Both **file1** and **file2** point to the **same inode**, so the link count is 2 for both. There is only one real file with two names.

What you do	Result
Change the content using file1	file2 shows the same change (it is the same data).
Delete file1 (rm file1)	file2 still works. Link count goes down to 1.
Delete both names	Link count becomes 0, and Linux frees the data on disk.

Command: ls -li (see the inode number)

```
$ ls -li
1048601 -rw-r--r-- 2 user group 6 Dec 18 10:00 file1
1048601 -rw-r--r-- 2 user group 6 Dec 18 10:00 file2
```

The first number is the **inode number**. Same number = same file. (Your number will be different.)
The option **-i** means **inode**.

Extra: `stat file1` shows the file details, including Inode and Links, in a full report.

5. Hard Links vs Soft Links

Soft link (symbolic link)

Syntax: `ln -s target link_name` (**-s** means **symbolic/soft**)

```
$ echo "data" > original
$ ln original hardlink      # hard link
$ ln -s original softlink  # soft link
$ ls -li
1048650 -rw-r--r-- 2 user group 5 Dec 18 10:10 hardlink
1048650 -rw-r--r-- 2 user group 5 Dec 18 10:10 original
1048677 lrwxrwxrwx 1 user group 8 Dec 18 10:10 softlink -> original
```

Look at	What it tells you
hardlink and original have the same inode	They are the same file with two names (count = 2).
softlink has a different inode	It is a separate small file whose only content is the path 'original'.
l at the start and -> original	l means symbolic link. The arrow shows where it points.
Link count of softlink is 1	Soft links do not increase the original's link count.

Comparison table

Point	Hard link	Soft (symbolic) link
Inode	Same inode as the original	Its own, new inode
What it stores	Nothing extra: just another name	The path of the target

Point	Hard link	Soft (symbolic) link
Different file systems	Cannot cross file systems	Can cross file systems
Link to a directory	Cannot link to directories	Can link to directories
If original is deleted	Still works (data stays until count is 0)	Becomes broken (dangling link)
Link count	Increases the count	Does not change the count
Command	In original name	In -s original name

When to use: Use a **hard link** when you want a safe second name for the same data. Use a **soft link** when you want a shortcut, especially to a folder or to something on another disk.

6. Importance of sudo for Privilege Escalation

sudo means **superuser do**. It lets a permitted user run a command as another user, usually **root** (the administrator). It gives **limited and controlled** power: only for that one command.

Syntax: sudo [options] command

Why use sudo and not stay as root?

Reason	Simple meaning
Prevents accidents	You use root power only when needed, so a wrong command does less damage.
Audit trail	Linux records who ran which sudo command (on Ubuntu/Debian in /var/log/auth.log).
Flexible	The admin can allow one user to run only certain commands, not everything.

```
# Running a privileged command
$ sudo apt update
```

7. Regular Command vs sudo Command

Type	Meaning
Regular command	Works only within your own permission limits.
sudo command	Runs with higher power, so it can reach restricted places.

```
# Regular user command
$ ls /root
ls: cannot open directory '/root': Permission denied

# sudo command
$ sudo ls /root
[sudo] password for user:
<content of /root directory is shown>
```

Remember: sudo asks for **your own** password (not root's), and remembers it for a few minutes. Only users who are allowed in the sudo list can use it.

8. Giving a User sudo Power

On Ubuntu and Debian, members of the group named **sudo** are allowed to use sudo. So we add the user to that group (this uses what you learned in Day 9 and Day 10):

```
# Add user to sudo group
$ sudo usermod -aG sudo username

# Check
$ groups username
username : username sudo
```

Part	What it does
usermod	Modify an existing user.
-a	Append: keep the user's old groups.
-G sudo	Add the group named sudo.
username	The user who will get sudo power.

Different systems: On Red Hat, CentOS and Fedora, the group is called **wheel** instead of sudo. Use `sudo usermod -aG wheel username`. Also, the user must **log out and log in again** for the new group to start working.

Useful extra sudo commands

Command	Use
sudo -l	List what the current user is allowed to run with sudo.
sudo -u user command	Run a command as another user (not root).
sudo visudo	Safely edit the sudo rules file (/etc/sudoers). It checks for mistakes before saving.

9. Practice Lab (Do in Order)

1. `mkdir lab && cd lab` (create and enter a practice folder)
2. `echo "hello" > file1` (create a file)
3. `ls -l` (file1 has link count 1)
4. `ln file1 file2` (make a hard link)
5. `ls -li` (same inode number, link count 2)
6. `ln -s file1 soft1` (make a soft link)
7. `ls -li` (soft1 has its own inode and shows an arrow)

8. `rm file1` (delete the original name)
9. `cat file2` (still works: hard link survives)
10. `cat soft1` (error: broken soft link)
11. `mkdir -p d1/a d1/b` (-p creates parent folders too)
12. `ls -ld d1` (link count is 4 = 2 + 2 subfolders)
13. `cd .. && rm -r lab` (clean up)

10. Quick Revision Cheat Sheet

Command	What it does
<code>ls -l</code>	Show details, including link count (2nd column)
<code>ls -ld folder</code>	Show details of the folder itself
<code>ls -li</code>	Show inode numbers
<code>stat file</code>	Full details of a file, including inode and links
<code>mkdir name</code>	Create a folder
<code>ln file newname</code>	Create a hard link
<code>ln -s file newname</code>	Create a soft (symbolic) link
<code>sudo command</code>	Run a command as root
<code>sudo -l</code>	List your sudo permissions
<code>sudo usermod -aG sudo user</code>	Give a user sudo power (Ubuntu/Debian)

Quick fact	Value
New file link count	1
New empty folder link count	2
Folder link count formula	2 + number of subfolders
Hard link	Same inode, count goes up, cannot cross file systems or link folders
Soft link	New inode, stores a path, can cross file systems and link folders, breaks if the original is deleted

Practice questions

- What is an inode? What is a link count?
- Why does a new empty folder have link count 2?
- A folder has 5 subfolders. What is its link count?
- What happens to file2 if file1 is deleted: (a) when it is a hard link, (b) when it is a soft link?
- Write any three differences between hard and soft links.
- Why is sudo safer than working as root all the time?

- Write the command to give the user **ravi** sudo access on Ubuntu.

Summary: Conclusion: Link count shows how many names a file or folder has. Hard and soft links give files extra names or shortcuts, and sudo lets you do admin work safely and with a record.